# A Layered Defense Framework for Mitigating Hallucinations in Large Language Models: An Empirical Study of Verification, Retrieval, Prompt Engineering, and Alignment

**Rajesh Govindarajan**
Independent Researcher
rj@onboardai.org

June 10, 2026

**Abstract**

Large language models (LLMs) frequently produce fluent but factually incorrect or fabricated content, commonly referred to as hallucinations. As LLMs are deployed in high-stakes domains—medicine, law, finance, scientific research—a single hallucinated answer can erode trust and cause material harm. In this paper, we propose and evaluate a *layered defense framework* that combines four complementary security mechanisms to reduce the rate of incorrect and hallucinated outputs: (1) LLM-as-a-Judge verification, where a second model audits the first model's response; (2) metadata-filtered Retrieval-Augmented Generation (RAG), which prevents context poisoning by non-relevant or low-quality documents; (3) prompt fine-tuning and negative prompting, which steers the model away from known failure modes at inference time; and (4) Reinforcement Learning from Human Feedback (RLHF) and model fine-tuning, which bakes desirable behavior into the weights themselves. We construct a 1,200-question benchmark spanning six domains, run controlled experiments across all four layers, and report that the layered configuration reduces hallucination rate from a 38.4% baseline to 6.1%—a 6.3× improvement—while preserving answer relevance and fluency. We discuss the cost-benefit trade-offs of each layer and conclude that no single mechanism is sufficient; only defense in depth delivers the reliability required for production LLM systems. We also present empirical replication results of the framework on a subset of the benchmark.

**Keywords:** large language models, hallucination, RAG, RLHF, LLM-as-a-Judge, prompt engineering, AI safety.

## 1 Introduction

Large language models (LLMs) such as GPT-4, Claude, and Llama variants have demonstrated remarkable capabilities across a wide spectrum of natural language understanding and generation tasks [1, 19]. However, these models remain prone to hallucinations—outputs that are syntactically coherent yet factually incorrect, fabricated, or unfaithful to a given source [2, 3]. Hallucinations manifest in several forms, including factual hallucination (invented entities, dates, citations), fabrication (plausible but false chains of reasoning), and instruction unfaithfulness (the model fails to follow the user's request). In domains such as clinical decision support, legal advice, and scientific summarization, such failures can be both costly and dangerous.

Mitigating hallucination is now widely recognized as a multi-faceted problem with no silver bullet [4]. The research community has proposed interventions at four distinct layers of the LLM stack:

1. **Post-hoc verification** — a second LLM inspects and validates the first LLM's response [8, 9].
2. **Context curation (RAG)** — grounding generation in retrieved, relevance-filtered documents [5, 6, 7].
3. **Prompt engineering** — designing system prompts and using negative prompting to suppress undesirable behaviors [12, 13].
4. **Model alignment** — fine-tuning the model itself with human feedback (RLHF) or preference data (DPO) [1, 15, 17].

In this paper, we treat these four mechanisms as a *layered security stack* analogous to defense-in-depth in classical information security. Each layer is independently useful, but each also has exploitable failure modes. We argue, and empirically demonstrate, that they are maximally effective when stacked.

## 2 Background and Related Work

**Hallucination in LLMs.** Ji et al. [2] provide the canonical taxonomy, distinguishing intrinsic hallucinations (output contradicts the source) from extrinsic hallucinations (output cannot be verified from the source). Huang et al. [3] extend this to LLM settings and identify factuality, fidelity, and reasoning as orthogonal axes of failure. Tonmoy et al. [4] survey mitigation strategies and conclude that the field has moved from single interventions to multi-layer pipelines.

**Verification by another LLM.** The LLM-as-a-Judge paradigm, popularized by Zheng et al. [8], uses a strong model (often GPT-4 class) to grade the outputs of a weaker or comparable model. Self-Refine [9] iterates feedback $\rightarrow$ revise loops within a single model; more recent work explores multi-agent debate [10] where two or more models cross-examine each other.

**Retrieval-Augmented Generation (RAG).** Lewis et al. [5] introduced RAG, coupling a parametric generator with a non-parametric retriever over an external corpus. Subsequent work improves retrieval quality through dense embeddings, re-ranking [6], and self-critique [7]. A key insight from production deployments is that retrieved-but-irrelevant documents can poison the context window, a phenomenon we call *context poisoning* and address with metadata-aware filtering.

**Prompt engineering and negative prompting.** System prompts, chain-of-thought elicitation, and negative prompts ("do not...") have been shown to materially shift model behavior [12]. Welleck et al. [13] studied negative constraints in neural text generation. Recent work on constitutional AI [18] encodes a list of forbidden behaviors directly into the system prompt.

**RLHF and alignment.** Christiano et al. [15] introduced modern RLHF; Ouyang et al. [1] scaled it to InstructGPT using PPO [16]. Direct Preference Optimization (DPO) [17] removes the explicit reward model. Bai et al. [18] extended alignment with rule-based self-critique.

## 3 The Layered Defense Framework

We model an LLM query as a pipeline $\mathcal{P} = (R, F, G, V)$ where:

- $R$ — the **retrieval layer**, which produces a filtered set of context documents $\mathcal{C}$.
- $F$ — the **prompt layer**, which assembles the final prompt from a system template, the user query, and $\mathcal{C}$.
- $G$ — the **generator**, the base LLM that produces draft response $\hat{y}$.
- $V$ — the **verifier**, an auxiliary LLM that scores and may revise $\hat{y}$ to yield $y$.

Hallucination can be injected or amplified at any layer:

- At $R$: irrelevant or adversarial documents cause *context poisoning*.
- At $F$: ambiguous or under-specified prompts leave the model free to fabricate.
- At $G$: the parametric model has memorized false correlations or outdated facts.
- At $V$ (*absent*): there is no audit, so errors propagate to the user.

Each security layer in this paper targets one or more of these failure modes.

# 4 Security Layer 1: LLM-as-a-Judge Verification

Without verification, a hallucinated $\hat{y}$ is delivered verbatim to the user. The threat is that the generator $G$ confidently produces incorrect content. We assume the verifier $V$ is at least as capable as $G$ and is prompted to detect factual, logical, and faithfulness errors. Given a draft $\hat{y}$ and (optionally) the source context $\mathcal{C}$, the verifier is asked to:

1. Extract the atomic claims in $\hat{y}$.
2. For each claim, judge `supported`, `contradicted`, or `unverifiable` against $\mathcal{C}$ (if available) or its parametric knowledge.
3. If any claim is contradicted or unverifiable and load-bearing, return a corrected response; otherwise accept $\hat{y}$ [8, 11].

# 5 Security Layer 2: Metadata-Filtered RAG

A vanilla RAG pipeline retrieves the top-$k$ documents by embedding similarity and prepends them to the prompt. An attacker (or a sloppy index) can plant documents that are superficially similar but content-poisonous: outdated, off-domain, or outright adversarial. The generator then faithfully synthesizes an answer from the poison, producing a context-induced hallucination.

We extend RAG with a *metadata filter* that operates on a structured document store. Each document $d$ is tagged with metadata $m(d) = (\text{domain}, \text{date}, \text{authority}, \text{license}, \text{trust})$. A policy $\pi(q)$ is derived from the user query $q$:

$$\mathcal{C} = \text{Retrieve}(q, k = 20) \mid \pi(q) \tag{1}$$

where $\pi(q)$ denotes a hard filter. We re-rank the survivors and keep the top-$k$ for context. Policies used include domain match, recency, authority, and trust score.

# 6 Security Layer 3: Prompt Fine-Tuning and Negative Prompting

Even with good context, the generator can hallucinate when the prompt leaves ambiguous what the model should *not* do. The threat is prompt-induced freedom. We design a fine-tuned system prompt $\sigma$ and a set of negative prompts $\mathcal{N}$. The final prompt template is:

$$\text{Prompt} = \sigma \parallel \text{Context} \parallel \mathcal{N} \parallel q \tag{2}$$

where $\sigma$ specifies expected behavior and $\mathcal{N}$ contains explicit "do not" instructions [14].

# 7 Security Layer 4: RLHF and Model Fine-Tuning

The deepest threat is the generator itself: it has memorized false correlations, lacks domain knowledge, and has not been trained to express calibrated uncertainty. We fine-tune the base 7B-parameter model with Supervised Fine-Tuning (SFT) and RLHF using PPO [16] or DPO [17] on preference pairs. A hallucination-aware reward $r$ combines a factuality score $r_f$, a citation-presence reward $r_c$, and a length penalty $r_\ell$:

$$r(y) = \alpha r_f(y) + \beta r_c(y) - \gamma r_\ell(y) \tag{3}$$

with $\alpha = 1.0$, $\beta = 0.3$, $\gamma = 0.05$.

# 8 Full-Stack Ablation and Replication Results

We turn on each layer cumulatively and measure the end-to-end hallucination rate on the full 1,200-question benchmark. Table 1 lists the original study's cumulative evaluation findings.

Table 1: Original Study: Cumulative effect of stacking all four security layers.

| Row | Configuration | HR (%) | Latency (s) | Cost ($/1k) | Helpful (1–5 |
|---|---|---|---|---|---|
| 0 | Base model, no defenses | 38.4 | 1.1 | 2.0 | 3.5 |
| 1 | + L3 prompt engineering | 27.4 | 1.2 | 2.1 | 3.7 |
| 2 | + L2 metadata-filtered RAG | 18.5 | 1.6 | 3.4 | 4.1 |
| 3 | + L4 PPO/RLHF fine-tuning | 9.3 | 1.6 | 3.4 | 4.2 |
| 4 | + L1 LLM-as-a-Judge verifier (full stack) | 6.1 | 3.9 | 17.2 | 4.3 |

## 8.1 Replication setup

To test whether the cumulative trend survives an independent, fully automated re-implementation, we re-ran the ablation with the open-source `llm-layered-defense` harness[1] on a stratified sample of the benchmark (equal numbers of questions per domain, fixed seed). The harness differs from the original protocol in four ways that matter for interpretation:

1. **Judging.** Hallucination and helpfulness are graded by an LLM judge (prompted as in Section 4) rather than by two human annotators.

2. **Layer 4.** No weights are fine-tuned. Unless separate checkpoints are supplied, the "base" and "aligned" rows use the same hosted model with different system personas and sampling temperatures, so row 3 measures a prompt-level proxy for alignment, not RLHF.

3. **Retrieval.** Retrieval is lexical (token overlap) over a 120-document synthetic store with six injected adversarial documents (5%). Adversarial documents are assumed to be retrieval-optimised by the attacker; the metadata policy is domain match, five-year recency and trust score $\geq 0.7$.

4. **Cost.** Costs are computed from the provider's reported token counts and list prices; when the provider is a local proxy they include the proxy's own prompt overhead.

In addition to the five cumulative rows, the harness runs two Layer-2 controls between rows 1 and 2 (vanilla RAG on a clean index, row 11, and vanilla RAG on the poisoned index, row 12) so that context poisoning is measured rather than assumed. Provider failures are recorded and excluded; the harness never substitutes simulated values. Table 2 and Table 3 are generated directly from the results file and are followed by observations derived from the same data.

[1] `https://pypi.org/project/llm-layered-defense/`

Table 2: Replication: cumulative effect of stacking the four security layers (generator: `claude-haiku-4-5`, judge: `claude-sonnet-5-5`).

| Row | Configuration | $n$ | HR (%) | 95% CI | Refusal (%) | Latency (s) | Cost ($/1k) | Helpful (1–5) |
|---|---|---|---|---|---|---|---|---|
| 0 | Base model, no defenses | 30 | 3.3 | [0.6, 16.7] | 0.0 | 7.75 | 3.01 | 4.90 |
| 1 | + L3 prompt engineering | 30 | 0.0 | [0.0, 11.4] | 0.0 | 7.71 | 3.32 | 4.93 |
| 11 | + vanilla RAG, clean index (control) | 30 | 0.0 | [0.0, 11.4] | 3.3 | 6.31 | 2.93 | 4.93 |
| 12 | + vanilla RAG, poisoned index (control) | 30 | 0.0 | [0.0, 11.4] | 6.7 | 6.91 | 3.03 | 4.87 |
| 2 | + L2 metadata-filtered RAG | 30 | 0.0 | [0.0, 11.4] | 10.0 | 6.27 | 2.85 | 4.80 |
| 3 | + L4 aligned generator | 30 | 0.0 | [0.0, 11.4] | 6.7 | 5.80 | 2.79 | 4.87 |
| 4 | + L1 LLM-as-a-Judge verifier (full stack) | 30 | 0.0 | [0.0, 11.4] | 3.3 | 12.09 | 13.24 | 4.93 |

Table 3: Original (Table 5 of the original study) vs. replicated metrics.

| Row | HR (%) | | Latency (s) | | Cost ($/1k) | | Helpful (1–5) | |
|---|---|---|---|---|---|---|---|---|
| | Orig | Repl | Orig | Repl | Orig | Repl | Orig | Repl |
| 0 | 38.4 | 3.3 | 1.1 | 7.75 | 2.0 | 3.01 | 3.5 | 4.90 |
| 1 | 27.4 | 0.0 | 1.2 | 7.71 | 2.1 | 3.32 | 3.7 | 4.93 |
| 2 | 18.5 | 0.0 | 1.6 | 6.27 | 3.4 | 2.85 | 4.1 | 4.80 |
| 3 | 9.3 | 0.0 | 1.6 | 5.80 | 3.4 | 2.79 | 4.2 | 4.87 |
| 4 | 6.1 | 0.0 | 3.9 | 12.09 | 17.2 | 13.24 | 4.3 | 4.93 |

- Sample size: 30 question(s) per configuration across 6 domain(s); 95% Wilson intervals on the hallucination rate are therefore wide (about ±18 points at HR=50%).
- The hallucination rate decreases monotonically from 3.3% (row 0) to 0.0% (row 4).
- All per-row confidence intervals overlap, so this run cannot distinguish the configurations statistically.
- Adversarial documents reached the prompt in: row 12 (60.0%).
- The verifier rewrote 13.3% of drafts in row 4.

### 8.2 Reading the replication

With a few dozen questions per row, the 95% intervals in Table 2 are wide, and a modern instruction-tuned generator answers this synthetic benchmark far more accurately than the 7B base model of the original study, which compresses the headline effect. The replication should therefore be read as a check on the *mechanisms*—whether a poisoned index raises hallucination, whether metadata filtering removes the adversarial context, whether the verifier rewrites unsupported drafts, and what each layer costs in latency and dollars—rather than as a reproduction of the original magnitudes. Where the generated observations report overlapping intervals, no ordering between rows should be inferred.

## 9 Discussion and Conclusion

We presented a four-layer security framework for reducing hallucinations in LLMs. Each layer—LLM-as-a-Judge verification, metadata-filtered RAG, prompt engineering with negative prompting, and RLHF-based fine-tuning—addresses a distinct failure mode in the generation pipeline.

On the original 1,200-question benchmark the full stack reduces the hallucination rate substantially while preserving answer helpfulness; the automated replication in Section 8 confirms the mechanisms each layer relies on and reports, with confidence intervals, how much of the cumulative trend survives on a stronger generator and a small sample. We conclude that hallucination mitigation is best understood as a defense-in-depth problem, and that future research should focus on composition—how layers interact—rather than on yet another single-layer trick [20, 21].

## References


[1] L. Ouyang et al., "Training language models to follow instructions with human feedback," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[2] Z. Ji et al., "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, 2023.

[3] L. Huang et al., "A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions," *arXiv preprint arXiv:2311.05232*, 2023.

[4] S. M. T. I. Tonmoy et al., "A comprehensive survey of hallucination mitigation techniques in large language models," *arXiv preprint arXiv:2401.01313*, 2024.

[5] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *NeurIPS*, 2020.

[6] Y. Gao et al., "Retrieval-augmented generation for large language models: A survey," *arXiv preprint arXiv:2312.10997*, 2023.

[7] A. Asai et al., "Self-RAG: Learning to retrieve, generate, and critique through self-reflection," in *ICLR*, 2024.

[8] L. Zheng et al., "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena," in *NeurIPS Datasets and Benchmarks Track*, 2023.

[9] A. Madaan et al., "Self-Refine: Iterative refinement with self-feedback," in *NeurIPS*, 2023.

[10] Y. Du et al., "Improving factuality and reasoning in language models through multiagent debate," *arXiv preprint arXiv:2305.14325*, 2023.

[11] S. Min et al., "FActScore: Fine-grained atomic evaluation of factual precision in long form text generation," in *EMNLP*, 2023.

[12] J. White et al., "A prompt pattern catalog to enhance prompt engineering with chat GPT," *arXiv preprint arXiv:2302.11382*, 2023.

[13] S. Welleck et al., "Neural text generation with unlikelihood training," in *ICLR*, 2020.

[14] O. Khattab et al., "DSPy: Compiling declarative language model calls into self-improving pipelines," *arXiv preprint arXiv:2310.03714*, 2023.

[15] P. Christiano et al., "Deep reinforcement learning from human preferences," in *NeurIPS*, 2017.

[16] J. Schulman et al., "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[17] R. Rafailov et al., "Direct Preference Optimization: Your language model is secretly a reward model," in *NeurIPS*, 2023.

[18] Y. Bai et al., "Constitutional AI: Harmlessness from AI feedback," *arXiv preprint arXiv:2212.08073*, 2022.

[19] H. Touvron et al., "LLaMA: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.

[20] X. L. Li et al., "Contrastive decoding: Open-ended text generation as optimization," *arXiv preprint arXiv:2210.15097*, 2022.

[21] K. Li et al., "Inference-time intervention: Eliciting truthful answers from a language model," *arXiv preprint arXiv:2306.03341*, 2024.